

COMP0016: Systems Engineering

Final Prototype Deliverables and Assessment

Dr. Yun Fu

Prof. Dean Mohamedally

Prof. Graham Roberts

Prof. Ivana Drobnjak

UCL Department of Computer Science

COMP0016 Assessment Components Weighting

Component Title	Weight	Submission Deadline	Led by
Coursework (HCI)(group)	5%	Nov 2025	Dr Chris Evans
Presentation (pitch)(group)	5%	Jan 2026	Prof Dean Mohamedally, Prof Graham Roberts, Prof Ivana Drobnjak, Dr. Yun Fu
Website Report (Group with Individual Contribution)	40%	16:00 Friday 27 Mar 2026	Prof Dean Mohamedally, Prof Graham Roberts, Prof Ivana Drobnjak, Dr. Yun Fu
System Artefacts (Group with Individual Contribution)	50%	16:00 Friday 27 Mar 2026	Prof Dean Mohamedally, Prof Graham Roberts, Prof Ivana Drobnjak, Dr. Yun Fu

Example Websites

- <https://students.cs.ucl.ac.uk/2024/group1/>
- <https://students.cs.ucl.ac.uk/2024/group4/>
- <https://students.cs.ucl.ac.uk/2024/group14/>
- <https://students.cs.ucl.ac.uk/2024/group15/>
- <https://students.cs.ucl.ac.uk/2024/group28/>
- <https://students.cs.ucl.ac.uk/2023/group1/>
- <https://students.cs.ucl.ac.uk/2023/group23/>
- <https://students.cs.ucl.ac.uk/2020/group8/>
- <https://students.cs.ucl.ac.uk/2019/group4/>
- <https://students.cs.ucl.ac.uk/2018/group24/>

Website Structure

- Home
- Requirements
- Research
- Algorithms (if applicable)
- UI Design (if applicable)
- System Design
- Implementation
- Testing
- Evaluation
- Appendices
 - User manual and deployment manual
 - GDPR and Privacy of Data
 - Development Blog – External Link on a public site (**this should be updated every two weeks in term 2**)
 - Monthly Video

Website Submission

- You can use **any tools or frameworks** to develop your report website but the submitted website should be a static website that **only includes the HTML, CSS, JavaScript files**.
- **The examiners should not be required to install anything and can open the index.html file to navigate all the pages.**
- **Please ensure all the links work** – especially for the index.html page **to navigate all of your pages**. We won't open the html files that are not linked.

Home

- Project title
- An abstract should include three paragraphs and the following contents
 - Problem statement
 - Your solution
 - Achievement and impact
- A 8 minutes video to introduce the project and go through the finished functionalities
- The development team
 - (including the team photos where possible) name, email, role or main contribution of each member. The roles include client liaison, UI design, researcher, programmer, report editor, tester.
 - You are welcome to include interesting background about yourself, as well as your LinkedIn profiles if you wish.
- Project management
 - Gantt chart (from October 20 2025 to March 27 2026)

Issues on the Home Page

- The abstract is too brief and requires additional detail.
- The project background and motivation are not included.
- The video is missing.
- The provided video is too short and does not effectively showcase all the implemented features.
- The resolution of the live demo is too low, making it hard to see the interface and visuals clearly.
- The narration is difficult to hear due to loud background music or ambient noise.
- The video resolution is too low, making it hard to view the content clearly.
- The Gantt chart is displayed too small, making the text difficult to read.

Requirements

- Partner Introduction and Project background
- Project Goals
- Requirement Gathering
 - How did you collect the requirements?
 - Did you design any survey? How did you analyse the survey data?
- Personas
 - Typical users of your project
- Use Cases (if applicable)
 - Use case diagram
 - List of use cases
- MoSCoW Requirement List (Functional and Non-functional)
 - A table for function requirements
 - Another table for non-functional requirements: https://en.wikipedia.org/wiki/Non-functional_requirement. For example, performance, security, usability, open source, maintainability, extensibility, etc.

Issues on the Requirements Page

- The use case diagram is missing.
- The requirements are described too briefly, typically consisting of only two or three words. They should be expanded to provide sufficient detail and clarity.
- The submitted use case diagram does not adhere to the proper format and instead resembles a workflow diagram.

MoSCoW requirement list

- **Functional** requirements

ID	Requirements	Priority
1		Must have
2		Should have
3		Could have

- **Non-functional** requirements

ID	Requirements	Priority
1		Must have
2		Should have
3		Could have

Requirement Example

2.6.1 Functional Requirements: General

ID	Requirement	Priority
FR-G1	The app shall allow unregistered users to register an account.	Must have
FR-G2	The app shall allow registered users to log in to their accounts, and show that they are logged in.	Must have
FR-G3	The app shall display a 'Home' page and browsing pages 'Projects', 'News', and 'Events' that are accessible without login.	Must have
FR-G4	The app shall allow any user to watch the project video for a given project.	Must have
FR-G5	The app shall allow any user to view a news article for a given news item.	Must have
FR-G6	The app shall allow any user to view event details for a given event item.	Must have
FR-G7	The app shall display an 'About' page that introduces the web application; providing context and background.	Should have
FR-G8	The app shall allow the user to search and filter on browsing pages.	Should have
FR-G9	The app shall allow registered users to edit their account details.	Should have
FR-G10	The app shall allow registered users to upload and change their own profile picture.	Could have
FR-G11	The app shall provide users with the option to reset their password if forgotten.	Could have
FR-G12	The 'Home' page or landing page shall showcase recent projects, published news and live events.	Could have
FR-G13	The app may send email notifications for events such as welcome messages or password resets as part of engaging with users.	Won't have

Requirement Example Continue

Functional Requirements - Admin

ID	Functional Requirements - Admin	Priority
FR-A1	The app shall display all campaigns on the app, including those created by the Admin and all the Vendors, on the main page.	Must have
FR-A2	The app shall allow the Admin to view the details and data analytics of all marketing campaigns on the app.	Must have
FR-A3	The app shall allow the Admin to suspend a campaign upon requests from a Vendor.	Must have
FR-A4	The app shall allow the Admin to view a data analytics overview for all marketing campaigns, including the total number of views, the total number of clicks, the number of views over a period, the number of clicks over a period, budget remaining, and campaign time remaining.	Must have
FR-A5	The app shall allow the Admin to edit account privileges.	Must have
FR-A6	The app shall allow the Admin to edit all marketing campaigns.	Should have
FR-A7	The app shall allow the Admin to send and receive messages from Vendors.	Should have
FR-A8	The app shall notify the Admin of new messages from Vendors.	Should have
FR-A9	The app shall allow the Admin to deactivate/activate Vendor accounts.	Should have
FR-A10	The app shall allow the Admin to generate invitation codes required to register new Vendor accounts.	Should have
FR-A11	The app shall allow an Admin to create new Admin accounts.	Should have
FR-A12	The app shall notify Admins when a message sent to a Vendor is not seen for a certain amount of time.	Could have

Research

- Related Projects Review
 - Review at least 1 existing similar project if applicable
 - For each of the similar projects, please list the project name, main features, and what you can learn from this existing application.
- Technology Review
 - Please compare the possible solutions, describe what you choose, and explain why.
 - Please compare the possible devices (if applicable), describe what you choose, and explain why.
 - Please compare the possible algorithms (if applicable), describe what you choose, and explain why.
 - Please compare the possible programming languages, frameworks, libraries, APIs, describe what you choose, and explain why.
- A summary of your technical decisions
- References
 - IEEE style: reference definition needs the number

"Several recent studies [1, 4, 15, 22] have suggested that. . . ."

[1] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, 2nd ed. Reading, MA: Addison Wesley, 2003. [E-book] Available: Safari e-book.
 - IEEE citation reference definition
 - <https://iee-dataport.org/sites/default/files/analysis/27/IEEE%20Citation%20Guidelines.pdf>

Common Issues on the Research Page

- The Review section is quite brief and lack details and should be expanded.
- The organization of the section needs improvement, as the discussion transitions abruptly between topics.
- A comprehensive summary of the technologies employed has not been included.
- References are absent.
- References are not cited within the body of the text using IEEE style.

Algorithms (if applicable)

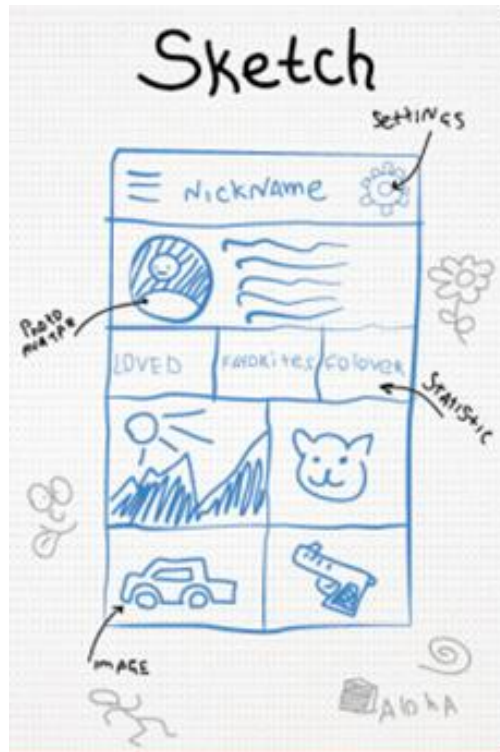
- Models
 - Describe the key idea of your chosen algorithms
- Data
 - Dataset
 - Data Preprocess (if applicable)
 - Training and testing sets
- Experiments
 - Experiment Design
 - Performance Evaluation Method (e.g. % accuracy)
 - Experiment results that should be presented by quantified values
 - Investigation of the optional Hyperparameters (if applicable)
 - Use plots or tables to show the performance comparison results
- Discussions
 - Why the algorithm fails for some test examples
 - Suggestions to improve the performance
- Conclusion
- References

Common Issues on the Algorithms Page

- The performance evaluation metric is not explained clearly.
- A quantitative evaluation has not been provided.
- No references are included.
- References are not cited within the text according to IEEE formatting requirements.

User Interface Design (if applicable)

- Design Principles (e.g. simplicity, consistency, visibility, feedback, tolerance, etc.)
- Hand-drawn sketches
- Online interactive wireframe URL (High Fidelity wireframe e.g. with Figma)



Wireframe



System Design

- System architecture diagram and a brief description of each component.
- Site Map (if applicable)
- Sequence Diagrams (if applicable)
- Design Patterns (if applicable)
- Class Diagrams (if applicable)
- Data storage (if applicable)
 - Please provide the data schema (e.g. ER diagram) if you have a database
- Packages and APIs defined (if applicable)

Common Issues on the System Design Page

- Several components are missing from the system architecture diagram.
- The ER diagram is not presented in the correct format and contains multiple inaccuracies.

Implementation (Very Important)

- Please describe how you implement the key features
 - It is very hard for the other people to understand your source code by only reading your code. You need to provide the necessary explanation documentation to help other people to understand how you implement the key features, for example, which frameworks or plugins or libraries you used for a particular feature and how you use them.
 - Please describe one key feature in a subsection. We take a website project as an example, you can describe the implementation details of the following features: authentication, database connection, data table, trend chart, csv data import, pdf generation, notification, etc.
 - If necessary, sequence or other diagram and short code snippet can be used to help the explanation.

Common Issues on the Implementation Page

- Some images fail to load.
- The system architecture diagram needs improvement. Some core components are missing.

Testing

- Testing Strategy
- Unit and Integration testing
- Compatibility testing (if applicable)
- Responsive design testing (if applicable)
- Performance/stress testing (if applicable)
- Prompt Testing (if applicable)
- User acceptance testing
 - Simulated testers
 - Test cases
 - Feedback from testers and project partners
- For each test, you should explain why you do this test, which test tool you use to conduct the test, what results you get, and your analysis or conclusion to the test

Common Issues on the Testing Page

- Details on how the tests were conducted are missing.
- Integration testing should be automated rather than relying solely on manual methods.
- Feedback from the project partner should be provided in the user acceptance testing section.

Evaluation

- Summary of achievements
 - An achievement table to list the MoSCoW functionalities, the completed states, and contributors
 - A list of known bugs (if applicable)
 - Individual contribution distribution table
- Critical evaluation of the project
 - User interface / user experience (if applicable)
 - Functionality
 - Stability
 - Efficiency
 - Compatibility
 - Maintainability
 - Project management
- Future work
 - How could the project be extended if you have more time

Common Issues on the Evaluation Page

- Some requirements were not implemented but has been incorrectly marked as completed.
- Separate individual contribution tables for the system development and the report website should be provided. Currently, only a combined table is included.
- The current "Future Work" section is quite brief. It would be better if it provided more detailed discussions on potential enhancements and next steps.

Example of Achievement Table and Known Bug List

- Achievement table

ID	Requirements	Priority	State	Contributors
1		Must	✓	All
2		Must	✓	All
3		Should	X	John
4		Could	X	Ali, Kai
Key Functionalities (must have and should have)		85% completed		
Optional Functionalities (could have)		65% completed		

- Known bug list

ID	Bug Description	Priority
1		High
2		Medium
3		Low

Individual Contribution Table For System Artefacts

Work packages	John	Ali	Kai
Research and Experiments	40%	10%	40%
UI Design (if applicable)	20%	70%	10%
Coding	40%	30%	30%
Testing	20%	0	80%
Overall contribution	30%	34%	36%

Individual Contribution Table For Website

Work packages	John	Ali	Kai
Website Template and Setup	0%	66%	34%
Home	33%	33%	34%
Video	33%	33%	34%
Requirement	34%	33%	33%
Research	50%	0	50%
Algorithm (if applicable)	20%	10%	70%
UI Design (if applicable)	20%	0	80%
System Design	20%	0	80%
Implementation	30%	35%	35%
Testing	0	0	100%
Evaluation and Future Work	60%	33%	33%
User and Deployment Manuals	20%	0	80%
Legal Issues	100%	0	0%
Blog and Monthly Video	33%	33%	33%
Overall contribution	30%	34%	36%

Appendices

- User manual
 - Teach users to use your application
 - It would be very helpful for the readers if some screenshots are provided to support your explanation.
 - If you need to deliver a website, please remember to provide the URL of the live website, the username, and password to login your website if authentication is required.
 - If there are more than one type of user (e.g. admin and normal user), please provide one account for each type of user.
- Deployment manual (**very important**)
 - This should provide a step by step guide to show your project partner how to deploy your project after they get your source code.
 - If needed for some difficult parts, please provide some screenshots.
- Legal issues and processes
 - Notes have been uploaded already to Moodle for you make use of in your project.
- Development Blog
- Monthly Video

Common Issues on the Appendices Page

- If applied, the URL of the deployed online application is missing and should be added.
- The test accounts are not provided.
- Screenshots should be added to support the user manual for clarity.

Legal Issues and Processes

- The notes for this section will be available in “Legal Implications - Data, IP and GDPR” section on Moodle.
- There is an extensive deck of notes for you to read through.
- Your website should include GDPR and privacy of data in your project; **please document this on your project website under Appendices.**
- Include a statement on the source code license (check with your Project Partners first) and GDPR of data governance.

Development Blog

- This is an open and public summary of your team's project. It should include your team websites' key assets, progress and experimental design. You can use WordPress, your own preferred blog, or any of the UCL blogging tools.
- The Project Partners have asked to keep track of your progress through the two terms. Ideas for the blog would be to include Interesting research, examples, reference materials, sketches (including from your HCI component), screenshots, algorithm code snippets especially what you have found and customised for the client's solution, and of course your own diagrams and models should be posted here.
- Check it with your Project Partners. You can post your team's development blog on social media with their emailed permission. Each student in a team should be able to contribute to this.
- As they are a diary, you should be aiming to update these blog entries every two weeks (do not submit it all at the end!).

Monthly Video

- Each month you should ideally be preparing a 1-2 minutes video update about interesting findings and progress made on your experiments.
- Use the PowerPoint PPTX template found in the “Project website, videos and blog” section as the start for your video (3-5 seconds).
- Please remember to set the OneDrive sharing setting of your video file to be ‘Anyone with the link’. If you need a date for your diaries to remember, do this by the 15th of each month.
- Please submit the OneDrive link on the form provided on Moodle and on the project website. UCL will put approved videos onto their YouTube channels which you can make use of.

Report Website Marking Criteria

- **Home:** Is the abstract explained well? How good is the video? How is the Gantt chart?
- **Requirements:** Is the project background and goals explained well? How well the project requirements are captured and described? Are personals and use cases described well? Are the MoSCoW requirements clear and specific?
- **Research:** How well the related project reviewed? How well you compared devices, tools, software, API, libraries, algorithms? Are references in place?
- **Algorithms** (if applicable): Is the model, experiments, results and discussion described well? Did you build up a quantified scheme to evaluate the performance?
- **UI Design** (if applicable): Is the UI design principals, sketches, prototypes documented well?
- **System Design:** Has the system architecture design been explained well? Quality of the ER diagram and suitable diagrams.

Report Website Marking Criteria Continue

- **Implementation:** How well is the implementation of key features described?
- **Testing:** Is there a good testing strategy and a thorough testing?
- **Evaluation:** Is there a good evaluation of the end results of the work? Are the criteria relevant, and the conclusions justified? Is the future plan well documented? Is the plan concrete?
- **Appendix:** Are the user and deployment manuals clear? Are legal issues, development blog, and month videos included?
- **Format:** Does the website have a good format?
- **Clarity:** Is the content well written and readable? How are the spelling and grammar of the report? Does it communicate effectively?

Mark Range for Report Website

Mark Range	Description	Criteria
90 - 100	Exceptional	Publishable quality. Close to faultless in documentation.
80 - 89	Outstanding	Could lead on to publishable work
70 - 79	Distinction	Very well written with a clear logical structure
60 - 69	Good (merit)	Clear project write-up with logical structure
50 - 59	Satisfactory (pass)	Adequate project write-up, lacking clarity or detail in places, or containing irrelevant material
40 - 49	Weak (BSc and MEng Pass)	Write-up is somewhat incoherent, rushed, contains important omissions, or irrelevant material
30-39	Inadequate(Fail)	Documentation is poor, unstructured, some parts missing.
0 - 29	Unacceptable fail	Documentation is substantially absent, badly written, incomprehensible or wrong.

Hosting the Project Website

- Hosting Space
 - Each team is assigned a 500 MB space for hosting their COMP0016 report website.
 - Your team's specific directory will follow the format **/cs/student/www/2025/group<i>/**, where <i> is your team number
 - Others can visit your project website using a URL that follows this pattern: **http://students.cs.ucl.ac.uk/2025/group<i>**